

NORTHERN UNIVERSITY
OF BUSINESS & TECHNOLOGY KHULNA
Knowledge for an Enlightening Future

DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING

Frontend Progress Report

ON

AI Programming Tutor & Bug Explainer

Course Code: CSE 4104

Course Title: Software Development III

SUBMITTED TO

Name: Md. Riaz Mahmud

Assistant Professor

Department of CSE

Northern University of Business & Technology Khulna.

SUBMITTED BY

Team Name	CSE4104 - 7B - T04
Section	7B
Project Title	AI Programming Tutor & Bug Explainer
Member Names	M.M. Asfy Or Rayhan Alisha Rahman Nadia Afrin Fatima Tanjim Rafa
Student IDs	11230121106 11230121112 11230121121 11230321556
GitHub Repository	https://github.com/Rayhan-55/cse4104-7b-t04-ai-programming-tutor-and-Bug-Explainer

Date of Submission: 06/07/2026

Team Information

Team Name	CSE4104 - 7B - T04
Section	7B
Project Title	AI Programming Tutor & Bug Explainer
Team Leader	M.M. Asfy Or Rayhan
Member Names	M.M. Asfy Or Rayhan Alisha Rahman Nadia Afrin Fatima Tanjim Rafa
Student IDs	11230121106 11230121112 11230121121 11230321556
GitHub Repository	https://github.com/Rayhan-55/cse4104-7b-t04-ai-programming-tutor-and-Bug-Explainer

Contents

1. **Frontend Technology Stack** ……..……...……..1

2. **Completed Pages** ……...……...……...……..4

3. **Backend APIs Integrated** ……...……...…….....…8

4. **Authentication Flow** ……...……...……...……...…12

5. **Current Development Progress** ……...……...……..14

6. **GitHub Repository Link** ……...……...……..17

Frontend Progress Report

Project Title: AI Programming Tutor & Bug Explainer

1. Frontend Technology Stack

The frontend of AI Programming Tutor & Bug Explainer is implemented as a single-page application (SPA). The following technologies, frameworks, and libraries were identified directly in the project's package.json and source files.

Core Framework and Build Tooling

The user interface is built using React version 18.3.1, a component-based UI library that is used to construct every page and component in the application. The project is bundled and served using Vite version 5.3.4, which also provides a development server configured with a proxy that forwards all /api requests to the backend running at http://localhost:5000, thereby avoiding cross-origin request issues during development. React Fast Refresh and JSX compilation are enabled through the official @vitejs/plugin-react plugin.

Routing and State Management

Client-side navigation is handled by React Router DOM version 6.25.1, which is used throughout the application via the BrowserRouter, Routes, Route, Navigate, and NavLink components to implement page routing as well as protected routes. Global authentication state, including the current user, a loading flag, and the login, register, and logout functions, is managed through a custom React Context provider named AuthContext, without relying on any external state-management library such as Redux.

HTTP Communication

All communication with the backend is performed through Axios version 1.7.2, configured as a single shared API client in src/api/client.js. This client uses a request interceptor that automatically

attaches the JWT bearer token to every outgoing request when the user is authenticated, removing the need to repeat this logic in individual pages.

Styling

The visual design of the application is implemented with Tailwind CSS version 3.4.7, a utility-first CSS framework that has been extended in tailwind.config.js with a custom color palette (including ink, panel, line, amber, teal, and muted tones) and custom font families, namely JetBrains Mono and Sora. PostCSS and Autoprefixer support this styling pipeline.

UI, Animation, and Data Visualization Libraries

Framer Motion version 11.3.8 is used to animate page transitions and staggered content reveals on the Landing, Dashboard, and GitHub Repo Explainer pages. Recharts version 2.12.7 is used to render bar charts summarizing AI tool-usage activity on both the Dashboard and the Admin Panel. The Monaco code editor, the same editor used in Visual Studio Code, is embedded through the @monaco-editor/react package version 4.6.0 to provide a proper code-editing experience on the Bug Explainer and Code Improver pages.

Browser-Native APIs

The application also makes use of the browser's native Web Speech API, specifically the SpeechSynthesis interface, through a custom VoiceButton component that reads AI explanations aloud as part of the Voice Tutor feature. This requires no external service or paid API and is available to every user.

Project Structure

The client codebase, located at codementor-ai/client, follows a clean and modular structure that separates concerns clearly. Page-level components live in src/pages, with one file per screen, including Landing, Login, Register, Dashboard, Tutor, BugExplainer, CompilerTranslator, CodeImprover, RepoExplainer, DocExplainer, and Admin. Reusable building blocks such as the Navbar, ProtectedRoute, CodeEditor, AIResultCard, Markdown renderer, and VoiceButton are placed in src/components. Global authentication state lives in src/context, while all backend API

calls are centralized and grouped by domain, namely AuthAPI, AiAPI, RepoAPI, DocAPI, and AdminAPI, inside src/api/client.js. Configuration files at the project root, including vite.config.js, tailwind.config.js, postcss.config.js, and a .env.example file for environment variables such as VITE_API_URL, complete the setup.

2. Completed Pages

The following pages exist as fully implemented React components and are registered as routes in src/App.jsx. No placeholder or unfinished page files were found in the project beyond a single generic fallback route that displays a "Page not found" message for unmatched paths. Each page is designed with reusable components to maintain a consistent user interface and navigation experience throughout the application. The pages are connected with React Router to support smooth client-side navigation without reloading the browser. Protected pages such as the Dashboard, Profile page, and Admin Panel are accessible only to authenticated users based on their role and authorization status. The implemented pages also communicate dynamically with backend APIs to display user data, AI responses, activity history, and administrative information.

Landing / Home Page

Route: /

This is the public marketing page that introduces the platform to first-time visitors. It presents a hero section, a simulated terminal example demonstrating the Compiler Error Translator, and a feature grid covering the Bug Explainer, Error Translator, Code Improver, AI Tutor, Voice Tutor, and the XP & Streaks system. It also includes call-to-action links directing visitors to register or to try the tools without creating an account, and uses Framer Motion for animated transitions.

Login Page

Route: /login

This page authenticates an existing user by collecting an email address and password. On submission, the credentials are sent to the backend, and upon a successful response the returned JWT is stored and the user is redirected to the Dashboard. Any error returned by the API, such as invalid credentials, is displayed inline to the user.

Registration Page

Route: /register

This page creates a new user account by collecting the user's name, email, and password. On successful registration, the user is automatically logged in as the returned token is stored immediately, after which the user is redirected to the Dashboard. Inline validation and error feedback are shown if registration fails.

Dashboard (Protected)

Route: /dashboard

This is the authenticated landing page shown after login. It displays the user's XP, level, streak, and total activity count, and renders a bar chart of the user's AI-tool usage built dynamically from their activity history retrieved from the backend. It also provides quick-access shortcuts to every AI tool and a list of the user's most recent activity.

User Profile Page (Protected)

Route: /profile

This page displays the authenticated user's name, email address, role, XP, level, and streak, together with a visual progress indicator. It also allows the user to update their name and, optionally, their password; the password field may be left blank to keep the existing password unchanged. Input is validated on the client side before submission, and the outcome of the update is communicated to the user through a toast notification.

AI Tutor

Route: /tutor

This is an AI interaction page where the user types a programming question, or selects one of several suggested example questions, and receives a beginner-friendly explanation generated by

the backend AI service. The page also offers a Voice Tutor option so the explanation can be read aloud.

Bug Explainer

Route: /bug-explainer

This page contains an embedded Monaco code editor into which the user pastes source code and selects its programming language. When submitted, the backend analyzes the code and returns a line-by-line explanation of the bug along with a suggested fix.

Compiler Error Translator

Route: /error-translator

This page allows the user to paste a raw compiler or runtime error message, or choose from a list of common example errors, and receive a plain-language explanation in both Bangla and English, making cryptic error messages easier to understand.

Code Improver

Route: /code-improver

This page also uses the embedded Monaco editor and allows the user to submit code for analysis. The backend returns suggestions covering cleaner variable naming, improved readability, and greater efficiency.

GitHub Repo Explainer

Route: /repo-explainer

This page accepts a public GitHub repository URL and retrieves both a plain-language project overview and a clickable file tree from the backend. Users may click on any file in the tree to receive an on-demand AI explanation of that specific file, and may choose the output language and use the Voice Tutor to hear the explanation.

PDF & Image Explainer

Route: /doc-explainer

This page supports drag-and-drop or click-to-upload of PDF documents and images, up to a maximum size of eight megabytes. Along with the uploaded file, the user may optionally add a specific question and select an output language, after which the file is sent to the backend as multipart form data and the returned explanation is displayed.

Admin Panel (Protected, Admin Only)

Route: /admin

This is an administrative dashboard restricted to users whose role is admin. It displays platform-wide statistics such as total users, admin count, total AI activity, and the top XP holder, along with a tool-usage bar chart, a searchable table of all users with controls to promote or demote their role, and a feed of recent activity across all users on the platform. A link to the Profile page is available from the Navbar for authenticated users, alongside links to the Dashboard and the relevant AI tool pages.

3. Backend APIs Integrated

The following API integrations were identified by analyzing the frontend API service file, src/api/client.js, together with the corresponding calls made from each page component. All of the endpoints listed below are called dynamically at runtime, and no static or hardcoded data was found substituting for these calls in any of the reviewed pages.

1. User Registration

POST /api/auth/register | Related Page: Registration Page

Creates a new user account with name, email, and password.

2. User Login

POST /api/auth/login | Related Page: Login Page

Authenticates a user and returns a JWT access token along with the user profile.

3. Get Current User

GET /api/auth/me | Related Page: AuthContext (app-wide, on load) / Dashboard

Verifies the stored JWT and retrieves the logged-in user's profile so the session can be restored automatically on page refresh.

4. Profile Update

PATCH /api/auth/profile | Related Page: User Profile Page

Submits an updated name and, optionally, a new password for the authenticated user's account. This endpoint is called by the frontend's Profile page; its availability on the backend should be confirmed independently, as backend controller code was outside the scope of this frontend-focused review.

5. AI Tutor Query

POST /api/ai/tutor | Related Page: AI Tutor Page

Submits a programming question and returns an AI-generated explanation.

6. Bug Explanation

POST /api/ai/bug-explainer | Related Page: Bug Explainer Page

Submits source code and its language for a line-by-line bug explanation and a suggested fix.

7. Error Translation

POST /api/ai/error-translator | Related Page: Compiler Error Translator Page

Submits a raw compiler or runtime error message for a plain-language translation.

8. Code Improvement

POST /api/ai/code-improver | Related Page: Code Improver Page

Submits source code for improvement suggestions relating to naming, structure, and efficiency.

9. AI Activity History

GET /api/ai/history | Related Page: Dashboard

Retrieves the authenticated user's past AI-tool activity to populate the usage chart and the recent-activity list.

10. Repository Overview

POST /api/repo/overview | Related Page: GitHub Repo Explainer Page

Fetches a public GitHub repository's metadata and file tree together with an AI-generated project overview.

11. Repository File Explanation

POST /api/repo/file | Related Page: GitHub Repo Explainer Page

Fetches a specific file from a repository and returns an AI-generated explanation of its contents.

12. Document/Image Explanation

POST /api/doc/explain | Related Page: PDF & Image Explainer Page

Uploads a PDF or image file as multipart form data, together with an optional question, and returns an AI-generated explanation.

13. Admin Statistics

GET /api/admin/stats | Related Page: Admin Panel

Retrieves platform-wide statistics including total users, admin count, activity count, top XP users, and usage broken down by feature.

14. Admin User List

GET /api/admin/users | Related Page: Admin Panel

Retrieves the full list of registered users for the admin user-management table.

15. Admin Recent Activity

GET /api/admin/activity | Related Page: Admin Panel

Retrieves recent AI-tool activity across all users for the admin activity feed.

16. Admin Role Update

PATCH /api/admin/users/:id/role | Related Page: Admin Panel

Promotes or demotes a user between the 'student' and 'admin' roles.

All AI-related endpoints, namely the tutor, bug explainer, error translator, code improver, repository overview and file explanation, and document explanation endpoints, are accessible without authentication. However, when a valid token is present, the backend's optionalProtect middleware automatically associates the activity with the logged-in user's account. This design allows guest users to try the tools freely while still enabling personalized history and XP accumulation for authenticated users.

4. Authentication Flow

Registration

The Registration page collects the user's name, email, and password and submits them to POST /api/auth/register. On a successful response, the returned JWT and user object are immediately stored, which effectively logs the user in without requiring a separate login step, and the user is then redirected to the Dashboard.

Login

The Login page collects the user's email and password and submits them to POST /api/auth/login. On success, the returned JWT and user object are stored, and the user is redirected to the Dashboard. If the login attempt fails, an inline error message sourced directly from the API response is displayed to the user.

JWT Token and Session Handling

Authentication in this project is stateless and entirely token-based. Upon a successful login or registration, the backend issues a JSON Web Token, which the frontend persists in the browser's localStorage under the key cm_token. An Axios request interceptor then automatically attaches this token as an Authorization: Bearer header on every outgoing API request. When the application first loads, AuthContext checks for a previously stored token and, if one is found, calls GET /api/auth/me to validate it and restore the user's session; if the token turns out to be invalid or expired, it is automatically removed from localStorage.

localStorage Usage

The only authentication-related value stored on the client side is the JWT itself, saved under the key cm_token. No passwords or other sensitive credentials are ever stored in the browser.

Protected Routes

The ProtectedRoute component wraps any route element that requires authentication. It reads the current user and a loading flag from AuthContext: while the session is still being verified it displays a loading indicator, an unauthenticated user is redirected to the Login page, and an authenticated user is granted access to the wrapped page. This mechanism is used to protect the Dashboard route.

Admin Protected Routes

The Admin Panel route reuses the same ProtectedRoute component but with an additional adminOnly flag. If the authenticated user's role is not admin, that user is redirected to the Dashboard instead of being shown the Admin Panel. This restriction is enforced on the frontend for navigation purposes and is independently re-enforced on the backend, where every route under /api/admin requires both a valid token, through the protect middleware, and an admin role, through the adminOnly middleware.

Logout

Logging out, which can be triggered from the Navbar, removes the JWT from localStorage and clears the in-memory user state, after which the user is redirected back to the Landing page.

Unauthorized Access Handling

On the frontend, any unauthorized attempt to access a protected page results in an automatic redirect to the Login page, or to the Dashboard in the case of a non-admin user attempting to reach the Admin Panel, rather than the page content being displayed. On the backend, requests to protected endpoints made without a valid token receive an HTTP 401 response, while requests to admin-only endpoints made by an authenticated but non-admin user receive an HTTP 403 response.

5. Current Development Progress

1. Completed Work

All twelve planned frontend pages of AI Programming Tutor & Bug Explainer — the Landing page, Login page, Registration page, Dashboard, User Profile page, AI Tutor, Bug Explainer, Compiler Error Translator, Code Improver, GitHub Repo Explainer, PDF & Image Explainer, and Admin Panel — have been implemented and are fully routed within the application. A shared library of reusable components, including the Navbar, ProtectedRoute, ConfirmDialog, CodeEditor, AIResultCard, Markdown renderer, and VoiceButton, supports consistent behavior and appearance across all of these pages. In addition, a centralized Axios-based API layer groups every backend call by domain, covering authentication, profile management, AI tools, repository exploration, document explanation, and administration.

2. API Integration Status

Sixteen backend endpoints spread across five route groups, namely auth, ai, repo, doc, and admin, have been integrated and are called dynamically from their respective pages. All data displayed on the Dashboard, Profile page, and Admin Panel, including statistics, charts, user lists, and activity feeds, is fetched from the backend at runtime rather than being hardcoded into the frontend. The Profile Update endpoint is called correctly from the frontend, though its implementation on the backend was outside the scope of this frontend-focused review and should be confirmed separately.

3. Authentication Status

A complete JWT-based authentication flow has been implemented and is fully functional, covering registration, login, automatic session restoration on page load, token attachment via an Axios interceptor, protected and admin-protected routing, logout with a confirmation step, and proper handling of unauthorized access through appropriate HTTP status codes returned by the backend.

4. Remaining Work and Areas for Further Testing

A number of areas remain for future development or additional testing based on what was found in the uploaded files. The project's own documentation describes the current build as a functional foundation rather than the fully realized platform that is envisioned, indicating that further planned features remain on a roadmap beyond what has been implemented so far. The Registration page does not currently include a confirm-password field, so a mistyped password is not caught at the point of entry. Confirmation dialogs are currently limited to the logout action and have not yet been extended to other sensitive operations, such as admin role changes. No automated test files or testing libraries were found in the project, so unit and integration test coverage does not yet exist. Finally, comprehensive end-to-end testing across different browsers and devices, along with a formal accessibility review, is not evidenced anywhere in the uploaded files and should be carried out before the project moves toward production deployment.

5. Form Validation

The application implements a shared, dependency-free validation utility located at src/utils/validation.js, which is reused across the Login, Registration, and Profile forms rather than being duplicated in each page. This utility checks that the name field is present and at least two characters long, validates the email field against a standard email-format pattern, and enforces a minimum password length of six characters. On the Profile page, the password field is treated as optional, so a user can update their name without being forced to also change their password. The Registration page additionally shows a live password-strength hint, labeled weak, okay, or strong, that updates as the user types. Beyond the authentication forms, the PDF & Image Explainer page validates the selected file's size on the client side, rejecting files larger than eight megabytes before they are sent to the backend. Validation errors are displayed as inline, field-level messages, and on the Login, Registration, and Profile pages they are additionally surfaced through toast notifications.

6. Responsive Design

The frontend uses Tailwind CSS responsive breakpoints, including sm, md, and lg, throughout the application. The Navbar collapses into a hamburger-style mobile menu below the medium

breakpoint, and the Landing page, authentication forms, Dashboard cards and charts, Profile page, and the AI tool pages adapt from a single-column layout on small screens to multi-column layouts on tablet and desktop screens. This responsive behavior is implemented directly in the component code; it should still be demonstrated with screenshots at submission, including at least one mobile-width view, as required by the assignment.

7. User Experience Improvements

The application provides a number of user-experience features found directly in the code. Loading indicators are shown while API requests are in progress on the Login, Registration, Profile, and AI-tool pages. A shared toast-notification system, implemented in src/context/ToastContext.jsx, displays success, error, and informational messages for actions such as login, registration, profile updates, and unauthorized access attempts, in place of plain inline text alone. Inline, field-level error messages accompany form validation failures. An empty-state message is shown on the Dashboard when a user has no activity history yet. A ConfirmDialog component requires the user to confirm before logging out, which helps prevent accidental sign-outs. Navigation is supported through the Navbar, protected and admin-protected routing, and quick-access shortcuts on the Dashboard.

Confirmation dialogs are currently limited to the logout action; extending this pattern to other sensitive operations, such as admin role changes, would further improve the user experience.

6. GitHub Repository Link

https://github.com/Rayhan-55/cse4104-7b-t04-ai-programming-tutor-and-Bug-Explainer